



**Fundação Educacional do Município de Assis
Instituto Municipal de Ensino Superior de Assis
Campus "José Santilli Sobrinho"**

YANN PEREIRA GARCIA

SIFEO: SISTEMA FINANCEIRO DE ESTOQUE E OPERAÇÕES

**Assis/SP
2026**



Fundação Educacional do Município de Assis
Instituto Municipal de Ensino Superior de Assis
Campus "José Santilli Sobrinho"

YANN PEREIRA GARCIA

SIFEO: SISTEMA FINANCEIRO DE ESTOQUE E OPERAÇÕES

Projeto de pesquisa apresentado ao curso de Análise e Desenvolvimento de Sistemas do Instituto Municipal de Ensino Superior de Assis – IMESA e a Fundação Educacional do Município de Assis – FEMA, como requisito parcial à obtenção do Certificado de Conclusão.

Orientando(a): Yann Pereira Garcia
Orientador(a): Douglas Sanches Cunha

Assis/SP
2026

FICHA CATALOGRÁFICA

Pereira Garcia, Yann.

SIFEO: Sistema Financeiro de Estoque e Operações / Yann Pereira Garcia.
Fundação Educacional do Município de Assis –FEMA – Assis, 2026.

Número de páginas.

1. Palavra-chave. 2. Palavra-chave.

CDD:
Biblioteca da FEMA

SIFEO: SISTEMA FINANCEIRO DE ESTOQUE E OPERAÇÕES

YANN PEREIRA GARCIA

Trabalho de Conclusão de Curso apresentado ao Instituto Municipal de Ensino Superior de Assis, como requisito do Curso de Graduação, avaliado pela seguinte comissão examinadora:

Orientador: _____ Douglas Sanches Cunha

Examinador: _____

DEDICATÓRIA

asdasda

AGRADECIMENTOS

Agradeço aqueles que sabem que fizeram parte da trajetória de aprendizado ao longo dos 3 últimos anos de curso, aqueles que participaram com os sorrisos as críticas que participaram da construção dos conceitos e das ideias utilizadas neste trabalho.

RESUMO

Texto.

Palavras-chave:

ABSTRACT

Texto em inglês.

Keywords:

LISTA DE ILUSTRAÇÕES

Figura 1: Mapa Mental.....	29
Figure 2: Diagrama de Classe.....	32
Figure 3: Diagrama de Entidade e Relacionamento.....	33
Figure 4: Caso de Uso.....	34

LISTA DE TABELAS

Tabela 1: Cronograma.....	21
Tabela 2: UC01 – Realizar Login.....	35
Tabela 3: UC02 – Cadastrar novo usuário.....	35
Tabela 4: UC02 – Cadastrar novo usuário.....	36
Tabela 5: UC03 – Manter Propriedade.....	37
Tabela 6: UC04 – Manter Atividade.....	38
Tabela 7: UC06 – Alternar Status da atividade.....	38
Tabela 8: UC08 – Alternar Status de venda do equipamento.....	39
Tabela 9: UC09 – Manter Equipamento.....	40
Tabela 10: UC0 –.....	40
Tabela 11: UC10 – Manter Insumo.....	41
Tabela 12: UC0 –.....	42
Tabela 13: UC12 – Adicionar categoria.....	43
Tabela 14: UC13 – Editar categoria.....	43

LISTA DE ABREVIATURAS E SIGLAS

ORM Object Relational Mapping

UML Unified Modeling Language

DER Diagrama de Entidade e Relacionamento

MER Modelo Entidade Relacionamento

WIP Work in Progress

SGBDR Sistema Gerenciador de Banco de Dados Relacional

JVM Java Virtual Machine

RF Requisitos Funcionais

LISTA DE SÍMBOLOS

SUMÁRIO

1. INTRODUÇÃO.....	14
1.1. OBJETIVO.....	15
1.1.1. Objetivo Geral.....	15
1.1.2. Objetivo Especifico.....	15
1.2. JUSTIFICATIVAS.....	16
1.3. MOTIVAÇÃO.....	17
1.4. PERSPECTIVAS DE CONTRIBUIÇÃO.....	17
1.5. METODOLOGIA.....	18
1.5.1. Revisão Bibliográfica e Levantamento de Requisitos.....	18
1.5.2. Modelagem e Arquitetura do software.....	18
1.5.3. Desenvolvimento Orientado a Fluxo.....	19
1.5.4. Backend e Lógica de Negócio.....	19
1.5.5. Frontend e Experiência do Usuário.....	20
1.5.6. Ferramentas de Apoio e Modelagem.....	20
1.6. CRONOGRAMA.....	21
1.7. ESTRUTURA DO TRABALHO.....	22
2. FERRAMENTAS E TECNOLOGIAS.....	23
2.1. JAVA.....	23
2.2. SPRING BOOT.....	23
2.3. GIT.....	24
2.4. GITHUB.....	24
2.5. POSTGRESQL.....	24
2.6. FIGMA.....	25
2.7. ASTAH.....	25
2.8. POSTMAN.....	25
2.9. ANGULAR.....	26
3. FUNDAMENTAÇÃO TEÓRICA.....	26
3.1. O CENÁRIO DAS MICROEMPRESAS AGRÍCOLA E A TECNOLOGIA.....	26
3.2. A ARQUITETURA DE INFORMAÇÕES E A GESTÃO DOCUMENTAL.....	26
3.3. DECISÕES ARQUITETURAIS E O STACK TECNOLÓGICO.....	27

4. ANÁLISE E ESPECIFICAÇÕES DO SISTEMA.....	28
4.1. MAPA MENTAL.....	28
4.2. REQUISITOS.....	29
4.2.1. Requisitos Funcionais.....	30
4.2.2. Requisitos Não Funcionais.....	30
4.3. DIAGRAMA DE CLASSE.....	31
4.4. DIAGRAMA DE ENTIDADE E RELACIONAMENTO.....	32
4.5. DIAGRAMA DE CASO DE USO.....	33
4.6. NARRATIVAS DE CASOS DE USO.....	34
4.6.1. UC01 – Realizar Login.....	35
4.6.2. UC02 – Cadastrar novo usuário.....	35
4.6.3. UC03 – Editar Perfil.....	36
4.6.4. UC04 – Manter Propriedade.....	36
4.6.5. UC05 – Manter Atividade.....	37
4.6.6. UC06 – Alternar Status Atividade.....	38
4.6.7. UC08 – Alternar Status de venda do equipamento.....	38
4.6.8. UC09 – Manter Equipamento.....	39
4.6.9. UC09 – Manter Setor.....	40
4.6.10. UC10 – Manter Insumo.....	41
4.6.11. UC11 – Manter Funcionário.....	41
4.6.12. UC12 – Adicionar Categoria.....	42
4.6.13. UC13 – Editar Categoria.....	43
5. DESENVOLVIMENTO DO PROJETO.....	44
6. CONCLUSÃO.....	44
7. REFERÊNCIAS.....	45
8. GLOSSÁRIO.....	49
9. APÊNDICE.....	50
10. ANEXOS.....	51

1. INTRODUÇÃO

No Brasil as pequenas e médias empresas agrícolas sempre foram vitais para a economia local e regional brasileira, como enfatiza Santos (2022, p. 8), mas frequentemente enfrentam desafios de gestão devido à falta de ferramentas informatizadas e específicas para o setor. Conforme Santos (2022, p. 11), o controle de entrada e saída de insumos, dentre eles os fertilizantes, sementes, defensivos e produtos colhidos, como grãos, frutas e hortaliças são pontos essenciais que, ao serem mal gerenciados, levam ao desperdício, perdas financeiras e dificuldades na tomada de decisão.

Conforme aponta Ignácio (2012, p. 184), a gestão e a análise estatística de dados são fundamentais para encontrar métodos eficientes que promovam o aumento da eficiência na produção, a redução de perdas e a consolidação de um histórico de dados confiável. Essa aplicação não se restringe apenas ao setor agrícola, estendendo-se a diversos outros campos, como medicina ciência, indústria e tecnologia. Portanto, conclui-se que para uma melhor gestão empresarial, é crucial processar os dados brutos, transformando-os em estatísticas e conhecimento estratégico. Tal processo, conforme destacado por Neto, Pinheiro e Coelho (2007, p. 11), é a chave para o avanço de qualquer microempresa. Isso demonstra o potencial impacto transformador do desenvolvimento de um software focado na gestão e análise de dados para microempresário agrícola.

Como destacado por Cunha (2022 p. 14-15), torna-se necessário capacitar as microempresas agrícolas e diminuir sua resistência ao uso de novas tecnologias. Sendo assim, o presente trabalho propõe o desenvolvimento de um software focado no microempreendedor que seja informal e eficiente, como o Sistema Financeiro de Estoque e Operações (SIFEO), podendo contribuir para uma gestão otimizada e um aumento na eficiência operacional. Nesse contexto, o desenvolvimento de um sistema de gerenciamento específico para microempresas agrícolas se justifica por sua relevância social e econômica buscando oferecer uma solução de baixo custo e alta aplicabilidade para um nicho que sustenta a economia rural, mas que geralmente não tem acesso a softwares ERP complexos e caros.

Tendo em vista estes aspectos, ao aplicar estes conhecimentos de engenharia de software ou sistemas de informação, podemos focar em usabilidade e funcionalidades essenciais na gestão de estoque e rastreabilidade básica. Por fim, com o controle apurado

do estoque de insumos evita a falta ou o prejuízo advindo da compra, enquanto o registro da saída de produtos facilita a rastreabilidade e a análise de rentabilidade por safra ou talhão.

Este trabalho está estruturado da seguinte maneira: a introdução irá apresentar a relevância do tema apresentado e seu contexto. Em sequência, serão mostrados os objetivos gerais e específicos. Usando a justificativa e a motivação será detalhado a proposta. A metodologia irá descrever quais foram os procedimentos adotados. O cronograma será utilizado para organizar todas as etapas do projeto, por fim, as referências bibliográficas que foram utilizadas.

1.1. OBJETIVO

1.1.1. Objetivo Geral

O objetivo geral deste trabalho é desenvolver e implementar o Sistema Financeiro de estoque e operações (SIFEO). Este sistema web visa oferecer uma solução de gestão acessível e especializada, capacitando microempreendedores agrícolas, permitindo controlarem de forma eficiente os fluxos financeiros (receitas e despesas) e a movimentação de insumos ao longo de safras. A meta é automatizar o armazenamento de dados e prover estatísticas gerenciais que suportem a tomada de decisões estratégicas e a otimização da produção.

1.1.2. Objetivo Especifico

Para alcançar o objetivo geral, serão desenvolvidos e implementados os seguintes objetivos específicos e funcionalidades:

- **Implementar Módulos de Cadastro Inicial:** Permitir o registro e autenticação do usuário e o cadastro das propriedades (sítios ou talhões) a serem gerenciadas.
- **Gerenciar Fluxo de Insumos e Clima:** Desenvolver a funcionalidade de registro detalhado de entrada e saída de insumos, assim como o registro de

variáveis climáticas (temperatura, umidade, índice pluviométrico) para análise contextualizada da produção.

- **Automatizar o Cálculo de Aplicação de Produtos:** Criar uma funcionalidade que, com base na área de aplicação e na dosagem recomendada por metro quadrado (g/m²).
- **Implementar Consulta Histórica por Safra:** Possibilitar a consulta e a filtragem de todos os registros (insumos, gastos e clima) realizados em anos anteriores, organizando os dados de maneira comparativa por safra ou ano.
- **Análise de Desempenho Econômico:** Desenvolver uma funcionalidade para análise financeira, que possa determinar a partir das entradas e saídas de valores, se houve lucro ou prejuízo em um determinado período (safra/ano), quantificando o resultado econômico para a gestão.

1.2. JUSTIFICATIVAS

O desenvolvimento deste trabalho se justifica devido a lacuna tecnológica e principalmente devido a falta de gerencia que podem ser identificada no segmento de microempreendimentos agrícolas. Enquanto grandes holdings agrícolas dispõem de softwares robustos de gestão, o pequeno produtor necessita de ferramentas acessíveis e especializadas para o controle de suas variáveis produtivas e econômicas.

Conforme Batalha (2005, p.11) ressalta, a necessidade de softwares de gestão empresarial é evidente, e para buscarmos uma solução é vamos focar na multiplicidade de dados do setor agrônomo e, simultaneamente, na manutenibilidade e na análise dos registros financeiros.

Portanto, o SIFEO propõe-se como uma solução de baixo custo e alta relevância, profissionalizando a gestão do campo e transformando dados como insumos, condições climáticas e outros, em informações estratégicas que auxiliam diretamente na otimização de custos, na previsão de necessidades e consequentemente na melhoria da competitividade do microempreendedor.

1.3. MOTIVAÇÃO

A principal motivação deste trabalho reside no reconhecimento dos desafios enfrentados por microempreendedores agrícolas ao tentar armazenar e analisar dados de maneira não virtualizada. A evolução tecnológica no campo tem se concentrado largamente nos maquinários, gerando inestimável ganho de eficiência operacional. Contudo, essa evolução exige, agora mais do que nunca, a virtualização e a sistematização da maneira como os dados são tratados.

Podemos verificar a necessidade de profissionalização gerencial, onde o produtor precisa de uma ferramenta que permita uma gestão personalizada de suas produções. O SIFEO surge para preencher esta lacuna, oferecendo uma solução que não apenas registra, mas que também trabalha os dados de forma eficiente e segura, permitindo ao produtor realizar a gestão com base em informações concretas e assim, tomar decisões levem a uma melhor eficiência e a sustentabilidade.

1.4. PERSPECTIVAS DE CONTRIBUIÇÃO

O presente trabalho visa contribuir de forma efetiva com o setor agrícola principalmente com os microempreendedores, atacando diretamente as deficiências na gestão de informações. O sistema SIFEO oferecendo como algumas das contribuições:

- **Apoio à Decisão Estratégica:** O software tras dados brutos de forma inteligente, permitindo a análise de gastos, o cálculo de custos e o gerenciamento de dados a longo prazo, dados cruciais para a tomada de decisões.
- **Otimização Financeira:** Contribui para a diminuição do número de gastos desnecessários e a otimização do uso de insumos, por meio da funcionalidade de cálculo de produto e da análise de lucratividade.
- **Comparação Interna:** Possibilita ao proprietário comparar produções e desempenhos econômicos de anos anteriores com a situação atual, gerando novas ideias e permitindo a visualização clara do próximo passo, melhorando o ambiente de trabalho e o controle de estoque de produtos.

1.5. METODOLOGIA

A metodologia para a execução deste Trabalho de Conclusão de Curso será estabelecida em três fases, garantindo a fundamentação teórica, a precisão na modelagem do sistema e a eficácia no desenvolvimento do *software*. Esta abordagem combina a pesquisa aplicada com a agilidade do gerenciamento de projetos orientado a fluxo (*Flow-Based Development*).

1.5.1. Revisão Bibliográfica e Levantamento de Requisitos

A primeira fase consistirá em um estudo exploratório aprofundado, com o objetivo de construir o embasamento teórico e contextualizar o problema a ser resolvido. Serão abordados o setor do agronegócio, as especificidades do microempreendedorismo e os desafios de gestão rural, aliados aos princípios arquivísticos de governança documental e temporalidade. Esta etapa visa à consolidação dos conceitos que fundamentam o sistema proposto. Paralelamente, será realizado o levantamento de requisitos junto ao público-alvo para identificar as necessidades funcionais e não funcionais. O resultado desta fase será o Documento de Requisitos, que servirá como base estrutural para os fluxos de trabalho na fase de desenvolvimento.

1.5.2. Modelagem e Arquitetura do software

Nesta fase, o foco será a documentação e a representação formal da arquitetura e do comportamento do sistema. Será utilizada a Linguagem de Modelagem Unificada (UML) para a criação dos seguintes diagramas, com o auxílio do software Astah:

- Diagrama de Casos de Uso: Para mapear as interações de alto nível entre os usuários e o sistema.
- Narrativas de Caso de Uso: Para detalhar textualmente os passos e fluxos de cada funcionalidade essencial.
- Diagrama de Classes: Para representar a estrutura estática do backend, incluindo entidades de dados e seus relacionamentos.

- **Diagrama de Sequência:** Para detalhar o fluxo de interações e operações dentro das funcionalidades mais críticas.

Adicionalmente, o design da interface do usuário (frontend) será prototipado na ferramenta Figma, e a estrutura do banco de dados relacional será modelada por meio do DER, utilizando o DB Designer.

1.5.3. Desenvolvimento Orientado a Fluxo

O gerenciamento e a execução do desenvolvimento de software serão conduzidos pelo método ágil Kanban, adaptado para o modelo Flow-Based Development ou Desenvolvimento Orientado a Fluxo, conforme proposto por David J. Anderson (2012). Os cartões utilizados no Kanban representarão o ciclo de vida completo das funcionalidades, passando desde a etapa lógica até a interface, sendo organizado por meio das seguintes etapas:

- **Não Iniciado:** Contém os requisitos e fluxos de valor a serem desenvolvidos.
- **Em Progresso:** Assegura o foco nas tarefas que estão ativamente em desenvolvimento, limitando gargalos.
- **Aguardando:** Destinada a tarefas que possuem impedimentos técnicos ou dependências externas.
- **Teste e Validação:** Etapa de verificação do código, incluindo testes de API via Postman e homologação da interface.
- **Concluído:** Fluxo de valor entregue, integralmente funcional e validado.

1.5.4. Backend e Lógica de Negócio

A camada de back-end será a principal fonte para o processamento do sistema. Sendo assim, será utilizada a linguagem Java em conjunto com o framework Spring Boot. Essa combinação é reconhecida por sua estabilidade e alto desempenho, sendo responsável por gerenciar toda a lógica de negócio do projeto, incluindo o registro e a gestão dos dados.

Para a persistência dos dados, será empregado o PostgreSQL, um Sistema Gerenciador de Banco de Dados Relacional (SGBDR) de código aberto. A modelagem do banco de

dados será previamente estruturada por meio do Diagrama Entidade-Relacionamento (DER), utilizando a ferramenta DB Designer.

1.5.5. Frontend e Experiência do Usuário

Referente à interface do usuário (front-end), será utilizado o framework Angular para a construção da aplicação. O Angular facilita o desenvolvimento de aplicações web dinâmicas e modulares, permitindo que o front-end consuma a API RESTful do back-end.

O planejamento visual será detalhado através de protótipos de tela criados no Figma, garantindo que a interface e a experiência do usuário estejam alinhadas aos requisitos levantados antes da codificação do front-end.

1.5.6. Ferramentas de Apoio e Modelagem

O controle de qualidade e a gestão do código são essenciais. Serão utilizados o sistema de versionamento Git e a plataforma GitHub para o gerenciamento do código-fonte, facilitando o trabalho individual e garantindo um histórico completo de todas as alterações.

Para a validação e os testes da API RESTful será utilizado o Postman, que permite a execução de testes de integração e a checagem da performance dos endpoints. Por fim, a modelagem formal do sistema será realizada utilizando a UML com o auxílio do software Astah, para a criação de diagramas estruturais e comportamentais, garantindo uma documentação de qualidade e precisa.

O uso integrado dessas tecnologias, orquestrado pelo método Kanban orientado a fluxo, garante que cada funcionalidade seja desenvolvida e validada, desde o banco de dados até a visualização final pelo usuário.

1.6. CRONOGRAMA

O cronograma de um projeto é uma das etapas fundamentais para o seu gerenciamento eficaz, conforme estabelecido pelo PMBOK (PMI, 2021), sendo indispensável para modelar e controlar a execução das atividades, incluindo durações e dependências. Para a concretização o acompanhamento visual desse modelo de software, foi adotado o Diagrama de Gantt. Este diagrama é uma representação gráfica que traduz a complexidade do cronograma em um formato de barras horizontais, facilitando a visualização entre os stakeholders e permitindo que a execução do software permaneça alinhada com o planejamento. O desenvolvimento desta ferramenta, historicamente atribuída a Henry Gantt (ca. 1910), e documentado por Clark (1922), descreve o sistema de gráficos criado para medir o desempenho de trabalho e comparar o que foi planejado com o que foi realizado, conforme podemos visualizar na Tabela:

Atividade/Mês	JAN	FEV	MAR	ABR	MAI	JUN	JUL	AGO	SET
Levantamento de referências bibliográficas	■								
Levantamento de requisitos do sistema	■	■							
Modelagem dos Diagramas do Sistema		■	■						
Escrita da Qualificação			■	■					
Exame de Qualificação			■	■					
Análise de prioridade dos fluxos				■					
Implementação do banco de dados				■	■				
Desenvolvimento dos fluxos de cadastro do usuário				■	■	■			
Desenvolvimento dos fluxos de Insumos					■	■	■		
Desenvolvimento dos fluxos de relatórios						■	■	■	
Testes do Sistema							■	■	
Escrita da versão final do TCC								■	■
Defesa									■

Tabela 1: Cronograma

1.7. ESTRUTURA DO TRABALHO

2. FERRAMENTAS E TECNOLOGIAS

Para que fosse possível realizar o desenvolvimento do sistema financeiro de estoque e operações (SIFOE) foi necessário a escolha de diversas tecnologias e ferramentas visando buscar a qualidade, manutenabilidade, escalabilidade, estrutura e eficiência do sistema. Analisaremos a seguir cada uma das tecnologias utilizadas.

2.1. JAVA

A escolha da linguagem Java para o desenvolvimento do *back-end* fundamenta-se em sua robustez, escalabilidade e maturidade tecnológica. Segundo a JetBrains (2020), desenvolvedora de ferramentas líderes da indústria de software, o ecossistema Java oferece um equilíbrio entre desempenho e produtividade, sustentado por *frameworks* consolidados que facilitam a construção de sistemas complexos. Esta eficiência é advinda de e viabilizada pela Java Virtual Machine (JVM), cuja arquitetura permite a execução dinâmica e otimizada de aplicações. Conforme analisam Sarimbekov et al. (2013), as características da JVM fornecem um ambiente controlado e adaptável para linguagens dinâmicas, garantindo não apenas a portabilidade entre diferentes plataformas, mas também uma gestão de recursos que favorece a estabilidade do sistema em tempo de execução.

2.2. SPRING BOOT

Complementando a linguagem Java, torna-se indispensável a utilização do *framework* Spring Boot pela sua capacidade de acelerar a entrega dos projetos. Segundo Galindo Junior, Rocha e Souza (2021), a adoção desta ferramenta é estratégica para o fluxo de trabalho acadêmico e profissional, pois permite automatizar tarefas repetitivas de configuração. Vale destacar sobretudo a produtividade proporcionada por ele, diminuindo o tempo e consequentemente os custos de desenvolvimento (GALINDO JUNIOR;

ROCHA; SOUZA, 2021), viabilizando a criação de sistemas complexos de forma mais ágil e eficiente.

2.3. GIT

Ao gerirmos o código-fonte e o controle de revisões, utilizou-se o Git, um sistema de controle de versão que garante a rastreabilidade e a integridade do projeto. Segundo Ponuthorai e Loeliger (2022), o Git permite o gerenciamento de fluxos de trabalho através de mecanismos de *branching* e *merging*, fundamentais para lidar com conflitos e manter a estabilidade do sistema. Ao aderirmos esta ferramenta asseguramos que o desenvolvimento ocorra de forma organizada e modular, permitindo a navegação entre diferentes estados do código.

2.4. GITHUB

Para auxiliar na gestão do ciclo de vida do software e controle de revisões, utilizou-se o sistema Git em conjunto com a plataforma GitHub. Conforme explicam Aquiles e Ferreira (2023), o Git permite manter um histórico detalhado da evolução do código, facilitando a reversão de alterações e a organização do trabalho. Ao complementar o GitHub como o serviço de hospedagem que provê a infraestrutura necessária para o armazenamento remoto e a colaboração, consolidando-se como uma ferramenta indispensável para o desenvolvimento de software moderno (AQUILES; FERREIRA, 2023)."

2.5. POSTGRESQL

Ao escolhermos algo para gerenciamento dos dados, optou-se pelo sistema de SGBDR PostgreSQL. A justificativa para a utilização desta ferramenta para o banco de dados foi devido a sua confiabilidade em ambientes críticos, sendo reconhecido como uma das tecnologias mais robustas. Assim como vemos em Conrad (2021), o PostgreSQL firmou sua relevância global se destacado como o 'Banco de Dados do Ano', evidenciando sua evolução constante e capacidade de oferecer alto desempenho e segurança na manipulação de informações complexas (CONRAD, 2021).

2.6. FIGMA

Para concepção das ideias visuais e à experiência do usuário, optou-se pelo Figma como ferramenta de prototipagem. De acordo com Silva, Iglesias e Boer (2024), o Figma permite a visualização e o teste das funcionalidades antes da implementação final do código. Possuindo a capacidade de garantir que o design das interfaces seja consistente e centrado no usuário, o Figma facilita o refinamento estético e funcional através de uma colaboração ágil entre os envolvidos no projeto (SILVA; IGLESIAS; BOER, 2024).

2.7. ASTAH

Segundo Rocha (2021), o Astah destaca-se como um recurso importante no processo de desenvolvimento de software, sendo fundamental para nas atividades de documentação. O uso do Astah nos permite organizar visualmente os requisitos do projeto, permitindo o melhor desempenho do desenvolvimento, possibilitando clareza na estrutura e no comportamento das funcionalidades do sistema (ROCHA, 2021).

2.8. POSTMAN

Como forma de validação e garantia da qualidade da API, utilizou-se a ferramenta Postman como recurso para testes de requisições. Segundo Westerveld (2021), o Postman permite testes para o melhor entendimento e funcionamento interno de uma API. Sua abordagem prática, que une teoria e exemplos do mundo real para garantir que as APIs sejam bem projetadas, documentadas e testadas antes de sua implementação final (WESTERVELD, 2021).

2.9. ANGULAR

Devido à sua estrutura robusta e modular, como vemos por Marinho e Garcia (2021), o uso de ferramentas *front-end* baseadas em componentes permite a criação de interfaces escaláveis e de fácil manutenção, estes benefícios nos trouxeram como principal opção o angular como *framework* para o *front-end*, onde cada elemento da tela é tratado de forma independente e reutilizável. A adoção desta tecnologia justifica-se pela sua eficiência em gerenciar estados complexos de aplicação, garantindo uma integração fluida com o *back-end* em Java e proporcionando uma experiência de usuário mais dinâmica (MARINHO; GARCIA, 2021).

3. FUNDAMENTAÇÃO TEÓRICA

3.1. O CENÁRIO DAS MICROEMPRESAS AGRÍCOLA E A TECNOLOGIA

O avanço da tecnologia da informação deixou de ser uma exclusividade das grandes corporações rurais e passou a ser uma necessidade vital para a agricultura. Segundo Cavaleiro *et al.* (2018), a adoção de sistemas de informação no agronegócio otimiza a tomada de decisão e a alocação de recursos financeiros e operacionais. Ao visualizar o contexto das microempresas agrícolas, o planejamento de safras e o controle de insumos frequentemente ocorrem por meios empíricos e manuais, a implementação de um software de gestão viabiliza a transição para um modelo melhor estruturado. Zambalde *et al.* (2011) corroboram essa visão ao afirmar que a informatização rural reduz desperdícios e aumenta a competitividade, transformando dados brutos coletados no campo em informações gerenciais estratégicas que garantem a sobrevivência e a escalabilidade do negócio.

3.2. A ARQUITETURA DE INFORMAÇÕES E A GESTÃO DOCUMENTAL

Para que os dados processados possuam uma maior valorização, o sistema deve garantir a rastreabilidade e a recuperação dessas informações de maneira eficiente. Para isso

uma estruturação por meio de nomenclaturas e nomeação técnica é fortemente aceita, sendo assim buscamos nos princípios da Biblioteconomia e da Arquivologia o entendimento destes conceitos para melhor aplicação no projeto. De acordo com Bellotto (2006), a classificação documental não deve focar no formato isolado do arquivo, mas sim refletir a sua função e a atividade que o gerou, garantindo a organização do conteúdo. Aplicando esse conceito ao software agrícola, os documentos produzidos como laudos agrônômicos, notas fiscais de insumos ou documentos locais são relacionados às suas determinadas rotinas de plantio ou gestão financeira. Para melhor comprovação de sua utilização vemos em outras formas de gestão que neste caso pode ser batizada por cartilhas de gestão documental (INSTITUTO FEDERAL DE SERGIPE, 2021), desta forma permite-se que o pequeno produtor possa localizar o histórico necessário da sua propriedade de forma intuitiva, mantendo a integridade do ciclo vital da informação e também sua padronização.

3.3. DECISÕES ARQUITETURAIS E O STACK TECNOLÓGICO

Do ponto de vista da Engenharia de *Software*, a viabilidade de um controle rigoroso de dados exige uma arquitetura que isole a complexidade operacional do produtor rural. A adoção do modelo cliente-servidor estabelece uma separação clara de responsabilidades, conforme ressalta Farias (2016). No lado do cliente (*Front-end*), a construção de uma interface dinâmica por meio de frameworks como Angular facilita a coleta de dados no ambiente rural. Simultaneamente, o *Back-end* operando em Java garante a robustez necessária para processar as regras de negócio. Diante deste cenário, Arantes (2001) destaca que a centralização em um servidor fortalece a proteção dos dados, essa arquitetura atinge sua máxima eficiência ao interagir com sistemas gerenciadores de banco de dados relacionais, como o PostgreSQL, que atua como base para manter o histórico dos dados das safras e a segurança simultânea das operações.

4. ANÁLISE E ESPECIFICAÇÕES DO SISTEMA

Este capítulo tem como objetivo detalhar a concepção técnica e a modelagem estrutural do software. Após o referencial teórico que fundamentou a viabilidade da arquitetura cliente-servidor e a aplicação de princípios arquivísticos para a organização da informação rural, a etapa atual concentra-se na leitura dessas necessidades de negócio em especificações de engenharia de software.

Para a construção do sistema, inicialmente apresentaremos a ideia inicial por meio do mapa mental prosseguindo para os requisitos funcionais e não funcionais, estabelecendo o escopo exato das operações de campo, controle financeiro e governança documental que a aplicação deve suportar. Em seguida, o comportamento e a estrutura do software são representados por meio de diagramas padrão da UML e pelo modelo relacional de banco de dados. Por meio destes tópicos veremos que eles garantem a integração entre a interface do usuário e a API de processamento de forma segura e operacional para o produtor rural.

4.1. MAPA MENTAL

A análise e especificação de requisitos necessita de uma compreensão profunda do problema antes da modelagem técnica estrutural. Sabendo disto, o mapa mental se apresenta como uma ferramenta estratégica para organizar as ideias iniciais, escopo geral do sistema e as funcionalidades. Conforme estabelecido por Buzan (2009), os mapas mentais utilizam uma estrutura que a partir de um núcleo central, permite categorizar e ramificar informações complexas de maneira lógica, intuitiva e hierárquica.

Funcionando como um instrumento facilitador na fase de elicitação de requisitos, o desenvolvimento do software de gestão para microempresas agrícolas foi facilitado com o uso do mapa mental, que permitiu abstrair as ideias e organizar melhor as funcionalidades. Segundo Pressman e Maxim (2016), o emprego de técnicas de modelagem visual nas fases iniciais é essencial permitindo alinhar a visão arquitetural com as reais necessidades dos usuários. Ao dividirmos de forma visual todo o corpo de uma propriedade rural, o mapa mental permitiu delimitar as fronteiras do sistema e

identificar quais são os requisitos funcionais principais, servindo como base sólida para a posterior elaboração dos artefatos da UML e do modelo relacional.



Figura 1: Mapa Mental

4.2. REQUISITOS

Ao avançarmos da ideia inicial do projeto, a análise avança para a fase de requisitos, esta etapa é fundamental o progresso do software. De acordo com Sommerville (2011), os requisitos de um sistema constituem na descrição detalhada dos serviços e de suas restrições operacionais. Contextualizando para o desenvolvimento ágil e estruturado, esta fase atua como um vínculo entre as necessidades do domínio de negócio, a gestão da microempresa agrícola e as definições da arquitetura de software, permitindo diminuir as ambiguidades e garantindo que as soluções finais ajudem o produtor rural.

Conforme descreve Paula Filho (2011), os Requisitos Funcionais (RF) definem o comportamento esperado do sistema e suas ações, trazendo as operações de campo, como o cadastro de setores, o controle de insumos e a submissão de documentos. Já os Requisitos Não Funcionais (RNF) estabelecem os critérios de qualidade, segurança e infraestrutura não tratando de funcionalidades específicas, mas impondo restrições de serviços, como por exemplo a utilização da linguagem Java, a persistência de dados em banco relacional PostgreSQL e as premissas de usabilidade da interface front-end.

4.2.1. Requisitos Funcionais

Os RF definem quais são as ações diretas que um determinado software executará para realizar as determinadas operações designadas. Conforme definido por Paula Filho (2009), esses requisitos descrevem o comportamento esperado do sistema e as transformações de dados em resposta às interações do usuário. No corpo deste projeto, as funcionalidades englobam o registro rotineiro de atividades de campo, como o cadastro de setores, o controle de insumos e a gestão de receitas e despesas. Além disso, ao aplicar os princípios arquivísticos abordados por Bellotto (2006), os RFs determinam que o sistema deve permitir o *upload* e a categorização de documentos e notas fiscais, vinculando esses documentos às suas determinadas entidades possibilitando garantir a rastreabilidade e a organicidade do acervo rural.

4.2.2. Requisitos Não Funcionais

Em contrapartida, os RNF estabelecem restrições tecnológicas, os critérios de qualidade e a infraestrutura necessária para a viabilidade do sistema. Segundo Sommerville (2011), esses requisitos não descrevem funcionalidades de negócio, mas impõem propriedades, segurança e padronização de arquitetura. Para a consolidação estrutural deste software, os RNF determinam qual será o modelo cliente-servidor, especificando o uso da linguagem Java no *back-end* para o processamento de regras e do *framework* Angular no *front-end* para garantir uma interface responsiva, para as restrições de armazenamento é necessário a persistência em um SGBDR PostgreSQL, garantindo a integridade do transporte de dados e a centralização segura de informações.

4.3. DIAGRAMA DE CLASSE

A transição dos requisitos funcionais para a arquitetura de um código necessita de uma estrutura rigorosa do domínio do problema. O Diagrama de Classes atua como um artefato estático da UML, fornecendo uma base para a programação orientada a objetos. Segundo Guedes (2011), este diagrama ilustra as classes que compõem o sistema junto com seus respectivos atributos e métodos, além disso ele permite mapear os relacionamentos lógicos. Essa visualização é essencial para diminuir o acoplamento excessivo e garantir que haja coesão das entidades na hora do desenvolvimento.

Contextualizando para softwares de gestão para a microempresa agrícola, o Diagrama de Classes foi concebido para espelhar de maneira fiel as operações e informações. De acordo com Larman (2007), a modelagem de domínio deve espelhar o vocabulário e os conceitos do mundo real. Dessa forma, entidades físicas e operacionais, como Sítio, Funcionários, Equipamentos e Insumos, devem ser modeladas com suas tipagens específicas. Ao lado desta ideia, para atender aos princípios arquivísticos, a classe Documento foi projetada como um eixo centralizador de informações. Ela estabelece relações de multiplicidade fundamentais com as origens geradoras dos dados, garantindo que a implementação no back-end suporte a rastreabilidade dos documentos e a sua coesão permitindo uma melhor aplicação do mapeamento objeto-relacional (ORM).

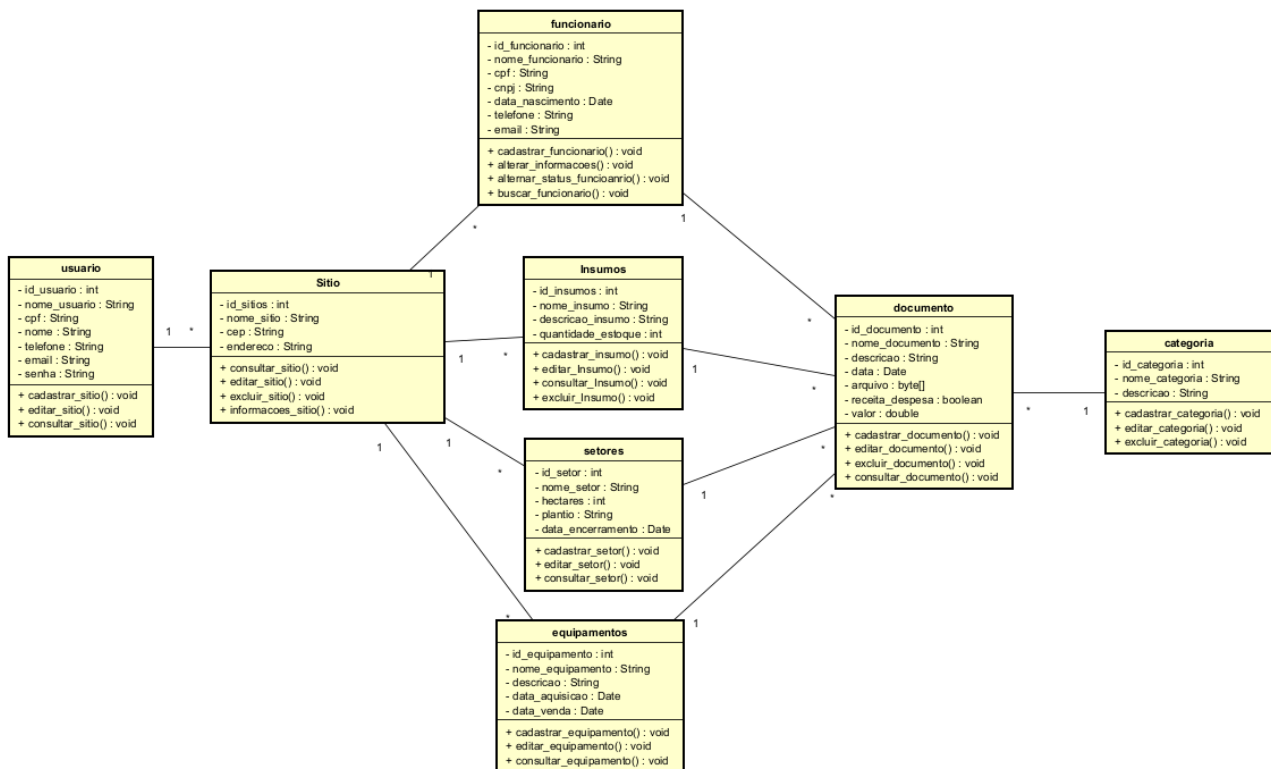


Figure 2: Diagrama de Classe

4.4. DIAGRAMA DE ENTIDADE E RELACIONAMENTO

A estruturação da camada de persistência de dados necessita de uma transição entre o paradigma orientado a objetos para o modelo relacional. O Modelo Entidade-Relacionamento (MER) e sua representação gráfica e o DER, constituem o alicerce para essa modelagem. De acordo com Heuser (2009), o objetivo dessa técnica é documentar de forma precisa as entidades e as restrições de integridade das informações que são acompanhadas do domínio de negócio. Enquanto o diagrama de classes fala sobre o comportamento das entidades com relação uma as outras, o DER concentra-se em como o SGBD armazenará e conectará as informações ao longo prazo. Elmasri e Navathe (2011) destacam que essa representação visual é fundamental para estabelecer chaves primárias, chaves estrangeiras e regras de cardinalidade que blindam o sistema contra anomalias de inserção e exclusão.

No desenvolvimento do sistema, o DER foi projetado com foco na integridade transacional e na documental centralizada. A modelagem reflete o mapeamento relacional das entidades, como Sítio, Funcionários, Insumos e Equipamentos, conectando-as à tabela

central de Documentos por meio de chaves estrangeiras, permitindo a rastreabilidade e a organização exigidas no levantamento de requisitos, somando a isso, o diagrama especifica os tipos dos dados voltados ao SGBD PostgreSQL.

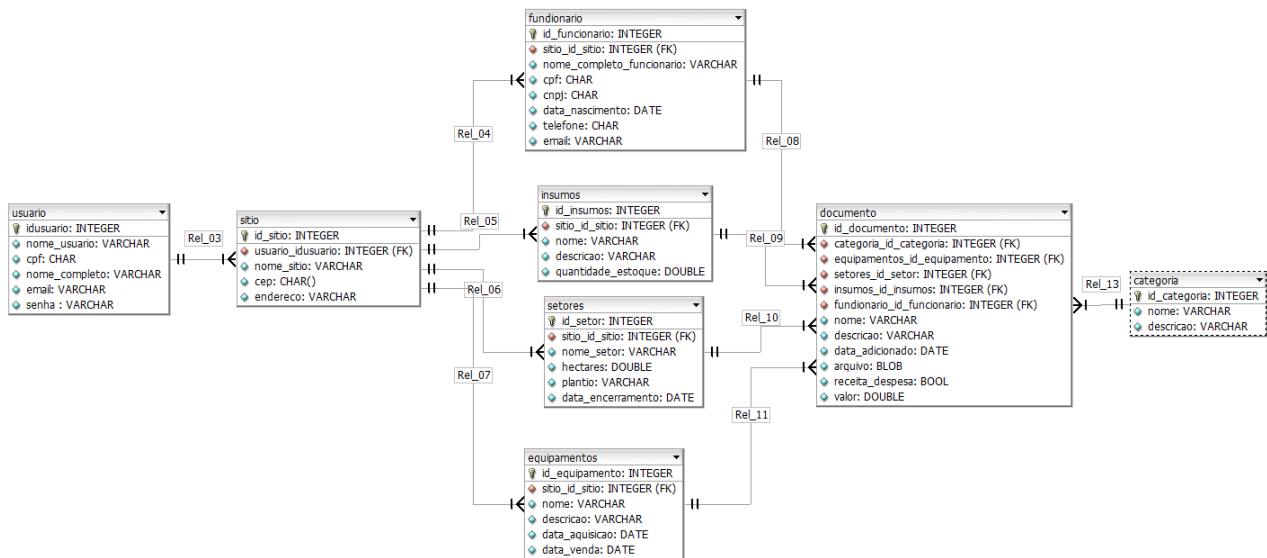


Figure 3: Diagrama de Entidade e Relacionamento

4.5. DIAGRAMA DE CASO DE USO

A união entre os usuários e o sistema pode ser formalizada através do Diagrama de Casos de Uso. Segundo Jacobson et al. (1999), pioneiros desta abordagem, os casos de uso mostram os requisitos funcionais na perspectiva de quem utiliza a aplicação, desta forma, definindo os limites do sistema e os atores que interagem com ele. Fowler (2004) complementa que este diagrama não tem o objetivo de detalhar a complexidade ou a lógica, mas sim mostrar o valor que o sistema entrega ao ser executado, atuando como uma ferramenta de comunicação entre a equipa de desenvolvimento e os *stakeholders* do projeto.

O Diagrama de Casos de Uso foi definido para mapear as ações da propriedade rural de forma lógica, o ator principal, que representa o produtor rural ou o gestor administrativo, estabelece interação direta com as funcionalidades centrais da aplicação, tais como o

registro de safras, a monitorização de insumos e a anexação de documentos legais. Visualizando as ideias de Cockburn (2001), possibilitou-se verificar que a modelação como "Anexar Documento" garantiria que cada caso de uso represente o cumprimento de um objetivo do domínio de negócio. Esta representação assegura que a rastreabilidade dos documentos e o controlo operacional sejam validados visualmente antes do desenvolvimento das interfaces no front-end.

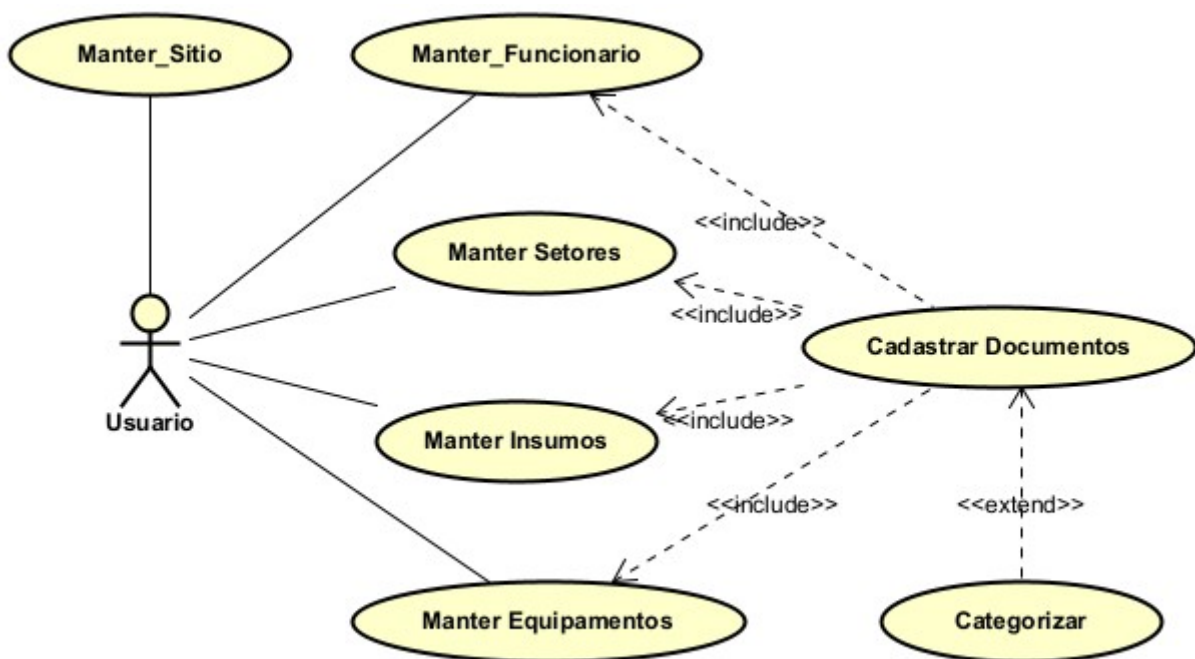


Figure 4: Caso de Uso

4.6. NARRATIVAS DE CASOS DE USO

O Diagrama de Casos de Uso fornece uma visão global das interações entre os atores e o sistema, mas ele é insuficiente para detalhar a lógica de negócio, as regras de validação e o fluxo de cada operação, então, a Engenharia de Requisitos recorre às Narrativas de Casos de Uso para somar a esta lacuna. Segundo Bezerra (2014), a descrição textual é o principal e o centro de onde se especificam as pré-condições, o evento principal e os fluxos alternativos ou de exceção, auxiliando na compreensão dos comportamento que o *software* deve apresentar em cada etapa.

4.6.1. UC01 – Realizar Login

1 - Finalidade / Objetivo	Verificar os usuários permitindo segurança dos dados e dos documentos.
2 - Atores	Usuários.
3 - Pré Condições	Usuário já cadastrados no aplicativo.
4 - Evento Inicial	Usuário deve acessar o caminho(URL) para acessar o sistema.
5 - Fluxo Principal	1. O usuário insere seu e-mail e sua senha. 2. O sistema verifica as informações digitadas no banco de dados. 3. Mensagem de “Sucesso.” (E01). 4. O sistema libera o acesso para a tela principal.
6 - Fluxo Alternativo	N/A
7 - Fluxos de Exceção	E01: Caso a informação digitada não corresponda aos dados do banco o sistema exibirá uma mensagem.

Tabela 2: UC01 – Realizar Login

4.6.2. UC02 – Cadastrar novo usuário

1 - Finalidade / Objetivo	Cadastrar a conta de um Usuário que ainda não possui vínculo com o aplicativo.
2 - Atores	Usuário
3 - Pré Condições	O usuário deve possuir um e-mail válido.
4 - Evento Inicial	Acessar a URL de cadastro.
5 - Fluxo Principal	1. O usuário clicará acessar a URL de cadastro. 2. O sistema solicitará algumas informações básicas. 3. Selecionar confirmar registro aparecerá uma mensagem “Cadastro bem-sucedido.”(E01). 4. Caso as informações sejam validas a conta será

	criada.
6 - Fluxo Alternativo	N/A
7 - Fluxos de Exceção	E01: Caso a informação digitada para criação de conta não sejam validas exibirá uma mensagem, “Dados inválidos tente novamente”.

Tabela 3: UC02 – Cadastrar novo usuário

4.6.3. UC03 – Editar Perfil

1 - Finalidade / Objetivo	Editar a conta do usuário.
2 - Atores	Usuário
3 - Pré Condições	O usuário deve possuir uma conta e estar autenticado.
4 - Evento Inicial	Acessar a URL de cadastro.
5 - Fluxo Principal	1. Clicar em seu perfil. 2. Selecionar editar perfil. 3. Alterar o formulário. 4. Confirmar alterações (E01).
6 - Fluxo Alternativo	N/A
7 - Fluxos de Exceção	E01: Informações invalidas: Retorna mensagem com a informação “Formulário possui informações inválidas”.

Tabela 4: UC02 – Cadastrar novo usuário

4.6.4. UC04 – Manter Propriedade

1 - Finalidade / Objetivo	Permite aos usuários cadastrar e gerenciar todos as propriedades que possui.
2 - Atores	Usuário.
3 - Pré Condições	Usuário já autenticado
4 - Evento Inicial	O usuário clica na aba lateral em propriedades.
5 - Fluxo Principal	1. Usuário clica em “Propriedades”. 2. Seleciona qual será a propriedade que deseja consultar as informações. 3. Exibirá as informações da propriedade (E01).
6 - Fluxo Alternativo	A01 – Cadastrar Propriedade: 1. Selecionar Registrar Propriedade. 2. Preencher o formulário. 3. Confirmar Registro. 4. O sistema envia uma mensagem de sucesso (E02). A02 – Excluir Propriedade: 1. Selecionar a propriedade que deseja excluir.

	2. Confirma ou não a ação. 3. O sistema envia uma mensagem de confirmação (E03). A03 – Editar Propriedade: 1. Selecionar a propriedade que deseja editar. 2. Preencher o formulário com as novas informações. 3. Confirmar alterações. 4. O sistema enviara uma mensagem de sucesso (E04).
7 - Fluxos de Exceção	E01 – Dados não encontrados: Retorna uma mensagem com a informação “Nenhuma informação encontrada” E02 – Nome já registrado: Nome da propriedade já existe (O sistema verificará propriedades com o mesmo nome). E03 – Impossível excluir propriedade: O sistema ira enviar uma mensagem “Impossível excluir, possui documento relacionado”. E04 – Impossível alterar informações: O sistema envia uma mensagem “Verifique os dados e tente novamente”.

Tabela 5: UC03 – Manter Propriedade

4.6.5. UC05 – Manter Atividade

1 - Finalidade / Objetivo	Realizar registro de atividade realizada em determinados setores.
2 - Atores	Usuário
3 - Pré Condições	Usuário já autenticado.
4 - Evento Inicial	Registrar uma atividade relacionada a um determinado setor.
5 - Fluxo Principal	1. Usuário clica em Manutenção. 2. Escolhe a aba de “Setores” ou “Equipamentos”. 3. Seleciona o setor que deseja visualizar. 4. Visualiza as informações (E01).
6 - Fluxo Alternativo	A01 – Cadastrar Atividade do setor: 1. Selecionar Registrar. 2. Preencher o formulário. 3. Confirmar Registro. 4. O sistema envia uma mensagem de sucesso (E02). A02 – Excluir Atividade: 1. Selecionar a propriedade que deseja excluir. 2. Selecionar se deseja confirma ou não a ação. 3. O sistema envia uma mensagem de confirmação (E03).

	A03 – Editar Atividade: 1. Selecionar a atividade que deseja editar. 2. Preencher o formulário com as novas informações. 3. Confirmar alterações. 4. O sistema enviara uma mensagem de sucesso (E04).
7 - Fluxos de Exceção	E01 – Dados não encontrados: Retorna uma mensagem com a informação “Nenhuma informação encontrada” E02 – Nome existente: Nome da atividade já existe (O sistema verificará atividade com o mesmo nome). E03 – Impossível excluir atividade: O sistema ira enviar uma mensagem “Impossível excluir, possui documento relacionado”. E04 – Impossível Alterar informações: O sistema envia uma mensagem “Verifique os dados e tente novamente”.

Tabela 6: UC04 – Manter Atividade

4.6.6. UC06 – Alternar Status Atividade

1 - Finalidade / Objetivo	Definir qual é o status atual da atividade.
2 - Atores	Usuário
3 - Pré Condições	O usuário deve estar autenticado e atividade não pode possuir status diferente de andamento.
4 - Evento Inicial	Usuário estar na tela de informações de Manutenção na aba de atividade.
5 - Fluxo Principal	1. Selecionar aba de Manutenção. 2. Selecionar a aba de plantio. 3. Selecionar “alterar status” da atividade desejada. 4. Escolher qual o novo status e salvar.
6 - Fluxo Alternativo	N/A
7 - Fluxos de Exceção	E01 - Impossível realizar operação: Mensagem devido ao status atual “Impossível realizar operação, atividade já finalizada”.

Tabela 7: UC06 – Alternar Status da atividade

4.6.7. UC08 – Alternar Status de venda do equipamento

1 - Finalidade / Objetivo	Definir por meio da data se o equipamento foi vendido ou não.
2 - Atores	Usuário
3 - Pré Condições	O usuário deve estar autenticado e o equipamento não pode possuir a data de vendido.
4 - Evento Inicial	1. Selecionar a aba de equipamentos.
5 - Fluxo Principal	1. Usuário deve estar na aba de Manutenção. 2. Selecionar a aba de equipamento. 3. Definir a data de venda. 4. Selecionar em confirmar operação (E01, E02).
6 - Fluxo Alternativo	N/A
7 - Fluxos de Exceção	E01 – Dado inválido: Mensagem informando erro, “Dado inválido, tente novamente.” E02 – Máquina já foi vendida: Retorno de mensagem informando, “Impossível realizar operação máquina vendida.”

Tabela 8: UC08 – Alternar Status de venda do equipamento.

4.6.8. UC09 – Manter Equipamento

1 - Finalidade / Objetivo	Permite aos usuários visualizar e gerenciar todos os equipamentos que possui.
2 - Atores	Usuários
3 - Pré Condições	Usuário já autenticado
4 - Evento Inicial	Selecionar a aba de equipamentos dentro de manutenção.
5 - Fluxo Principal	1. Selecionar a aba de equipamentos. 2. Selecionar qual equipamento deseja visualizar. 3. Novo dialog com as informações do equipamento (E01).
6 - Fluxo Alternativo	A01 – Cadastrar equipamento: 1. Selecionar Registrar. 2. Preencher o formulário. 3. Confirmar Registro. 4. O sistema envia uma mensagem de sucesso (E03). A02 – Excluir equipamento: 1. Selecionar o equipamento que deseja excluir. 2. Selecionar se deseja confirma ou não a ação. 3. O sistema envia uma mensagem de confirmação (E02, E04). A03 – Editar equipamento:

	1. Selecionar o equipamento que deseja editar. 2. Preencher o formulário com as novas informações. 3. Confirmar alterações. 4. O sistema enviara uma mensagem de sucesso (E03, E04).
7 - Fluxos de Exceção	E01 – Dados não encontrados: Retorna uma mensagem com a informação “Nenhuma informação encontrada” E02 – Impossível excluir equipamento: O sistema ira enviar uma mensagem “Impossível excluir, possui documento relacionado”. E03 – Erro ao inserir informações: O sistema envia uma mensagem “Verifique os dados e tente novamente”. E04 – Maquina já foi vendida: Retorno de mensagem informando, “Impossível realizar operação máquina já foi vendida.”

Tabela 9: UC09 – Manter Equipamento

4.6.9. UC09 – Manter Setor

1 - Finalidade / Objetivo	Permite aos usuários cadastrar e gerenciar todos os equipamentos que possuir.
2 - Atores	Usuários.
3 - Pré Condições	Usuário já autenticado possuir um sítio cadastrado.
4 - Evento Inicial	Selecionar aba de manutenção e de setores.
5 - Fluxo Principal	1. Selecionar a aba de setor. 2. Selecionar qual o setor que deseja visualizar. 3. Novo dialog com as informações do setor (E01).
6 - Fluxo Alternativo	A01 – Cadastrar Setor: 5. Selecionar Registrar. 6. Preencher o formulário. 7. Confirmar Registro. 8. O sistema envia uma mensagem de sucesso (E03). A02 – Excluir Propriedade: 1. Selecionar a propriedade que deseja excluir. 2. Confirma a ação. 3. O sistema envia uma mensagem de confirmação (E02). A03 – Editar Propriedade: 1. Selecionar a propriedade que deseja editar. 2. Preencher o formulário com as novas informações. 3. Confirmar alterações. 4. O sistema enviara uma mensagem de sucesso (E03).
7 - Fluxos de Exceção	E01 – Dados não encontrados: Retorna uma

	mensagem com a informação “Nenhuma informação encontrada” E02 – Impossível excluir atividade: O sistema ira enviar uma mensagem “Impossível excluir, possui documento relacionado”. E03 – Erro ao inserir informações: O sistema envia uma mensagem “Verifique os dados e tente novamente”.
--	--

Tabela 10: UC0 –

4.6.10. UC10 – Manter Insumo

1 - Finalidade / Objetivo	Permite aos usuários cadastrar e gerenciar todos os equipamentos que possuir.
2 - Atores	Usuários
3 - Pré Condições	O usuário deve estar autenticado
4 - Evento Inicial	Acessar a aba de insumos.
5 - Fluxo Principal	1. Selecionar a aba de Insumos. 2. Selecionar qual o insumo que deseja visualizar. 3. Novo dialog com as informações do insumo (E01).
6 - Fluxo Alternativo	A01 – Cadastrar Insumo: 1. Selecionar Registrar. 2. Preencher o formulário. 3. Confirmar Registro. 4. O sistema envia uma mensagem de sucesso (E03). A02 – Excluir Insumo: 1. Selecionar a propriedade que deseja excluir. 2. Confirma a ação. 3. O sistema envia uma mensagem de confirmação (E02). A03 – Editar Insumo: 1. Selecionar a propriedade que deseja editar. 2. Preencher o formulário com as novas informações. 3. Confirmar alterações. 4. O sistema enviara uma mensagem de sucesso (E03).
7 - Fluxos de Exceção	E01 – Dados não encontrados: Retorna uma mensagem com a informação “Nenhuma informação encontrada” E02 – Impossível excluir atividade: O sistema ira enviar uma mensagem “Impossível excluir, possui documento relacionado”. E03 – Erro ao inserir informações: O sistema envia uma mensagem “Verifique os dados e tente novamente”.

Tabela 11: UC10 – Manter Insumo

4.6.11. UC11 – Manter Funcionário

1 - Finalidade / Objetivo	Permite aos usuários cadastrar e gerenciar todos os funcionários.
2 - Atores	Usuários
3 - Pré Condições	O usuário deve estar autenticado e acessar a aba de funcionários.
4 - Evento Inicial	
5 - Fluxo Principal	1. Selecionar a aba de Funcionários. 2. Selecionar qual o funcionário que deseja visualizar. 3. Novo dialog com as informações do funcionário (E01)
6 - Fluxo Alternativo	A01 – Cadastrar Funcionário: 5. Selecionar Registrar. 6. Preencher o formulário. 7. Confirmar Registro. 8. O sistema envia uma mensagem de sucesso (E03). A02 – Excluir Funcionário: 1. Selecionar a propriedade que deseja excluir. 2. Confirma a ação. 3. O sistema envia uma mensagem de confirmação (E02). A03 – Editar Funcionário: 1. Selecionar o funcionário que deseja editar. 2. Preencher o formulário com as novas informações. 3. Confirmar alterações. 4. O sistema enviara uma mensagem de sucesso (E03).
7 - Fluxos de Exceção	E01 – Dados não encontrados: Retorna uma mensagem com a informação “Nenhuma informação encontrada” E02 – Impossível excluir atividade: O sistema ira enviar uma mensagem “Impossível excluir, possui documento relacionado”. E03 – Erro ao inserir informações: O sistema envia uma mensagem “Verifique os dados e tente novamente”.

Tabela 12: UC0 –

4.6.12. UC12 – Adicionar Categoria

1 - Finalidade / Objetivo	Permite aos usuários cadastrar uma nova categoria de documentos.
2 - Atores	Usuários
3 - Pré Condições	Usuários estar autenticado e estar na aba de categorias
4 - Evento Inicial	Selecionar a aba de categorias dentro de documentos.
5 - Fluxo Principal	1. Selecionar categoria. 2. Selecionar “Registrar.”. 3. Preencher o formulário. 4. Selecionar “Adicionar.” (E01).
6 - Fluxo Alternativo	N/A
7 - Fluxos de Exceção	E01 – Formulário inválido: Sistema ira retornar mensagem, “Verifique as informações e tente novamente”

Tabela 13: UC12 – Adicionar categoria.

4.6.13. UC13 – Editar Categoria

1 - Finalidade / Objetivo	Permite aos usuários editar as informações de uma categoria.
2 - Atores	Usuários
3 - Pré Condições	Usuários estar autenticado e estar na aba de categorias
4 - Evento Inicial	Selecionar a aba de categorias dentro de documentos.
5 - Fluxo Principal	1. Selecionar categoria. 2. Selecionar “Editar.”. 3. Preencher o formulário. 4. Selecionar “Adicionar.” (E01).
6 - Fluxo Alternativo	N/A
7 - Fluxos de Exceção	E01 – Formulário inválido: Sistema ira retornar mensagem, “Verifique as informações e tente novamente”

Tabela 14: UC13 – Editar categoria.

5. DESENVOLVIMENTO DO PROJETO

Para que fosse possível realizar o desenvolvime

6. CONCLUSÃO

7. REFERÊNCIAS

ANDERSON, David J. Kanban: Mudança Evolucionária de Sucesso para seu Negócio de Tecnologia. Porto Alegre: Bookman, 2012.

AQUILES, Alexandre; FERREIRA, Rodrigo. **Git e GitHub**: Controle de versão para desenvolvedores. São Paulo: Casa do Código, 2023. Disponível em: https://books.google.com.br/booksid=ShpTEQAAQBAJ&dq=github+plataforma+de+versionamento&lr=&hl=pt-BR&source=gbs_navlinks_s.

ARANTES, Juliana Amaral. **Modelo analítico para avaliar plataformas cliente/servidor e agentes móveis aplicado a gerência de redes**. 2001. Dissertação (Mestrado em Ciência da Computação) – Centro Tecnológico, Universidade Federal de Santa Catarina, Florianópolis, 2001. Disponível em: <http://repositorio.ufsc.br/xmlui/handle/123456789/80003>. Acesso em: 24 mar. 2026.

BATALHA, Mário Otávio; BUAINAIN, Antônio Márcio; SOUZA FILHO, HM de. Tecnologia de gestão e agricultura familiar. **Gestão Integrada da Agricultura Familiar. São Carlos (Brasil): EDUFSCAR**, p. 43-66, 2005.

BELLOTTTO, Heloísa Liberalli. **Arquivos permanentes**: tratamento documental. 4. ed. Rio de Janeiro: FGV, 2006.

BEZERRA, Eduardo. **Princípios de análise e projeto de sistemas com UML**. 3. ed. Rio de Janeiro: Elsevier, 2014.

BUZAN, Tony. **Mapas mentais**: métodos criativos para estimular o raciocínio e usar ao máximo o potencial do seu cérebro. Rio de Janeiro: Sextante, 2009.

CAVALHEIRO, Diego da Silva *et al.* A Tecnologia da Informação no Agronegócio: uma Revisão Bibliográfica. **Anais da Mostra de Iniciação Científica, Pós-graduação, Pesquisa e Extensão**, Caxias do Sul, v. 1, n. 17, 2018.

CLARK, Wallace. *The Gantt Chart: A Working Tool of Management*. New York: The Ronald Press Company, 1922. Disponível em: <https://archive.org/details/ganttchartworkin00claruoft/page/44/mode/2up>. Acesso em: 12/11/2025.

COCKBURN, Alistair. **Writing effective use cases**. Boston: Addison-Wesley, 2001.

CONRAD, A. Database of the Year: Postgres. *IEEE Software*, [S. l.], v. 38, n. 5, p. 130-132, set./out. 2021. Disponível em: <https://ieeexplore.ieee.org/abstract/document/9520330>. Acesso em: 19 fev. 2026.

CUNHA, E. M. da S. *Gestão e desenvolvimento nas pequenas propriedades rurais*. 2023. 43 f. Trabalho de Conclusão de Curso (Graduação em Administração) – Instituto Federal do Espírito Santo, Barra de São Francisco - ES, 2023.

ELMASRI, Ramez; **NAVATHE**, Shamkant B. **Sistemas de banco de dados**. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

FARIAS, Kleinner. **Arquitetura de software**. São Leopoldo: Unisinos, 2016. Disponível em: <https://kleinnerfarias.github.io/pdf/books/book-arqsoft-2016.pdf>. Acesso em: 11 mar. 2026.

FOWLER, Martin. **UML destilado**: um guia prático para a linguagem de modelagem unificada. 3. ed. Porto Alegre: Bookman, 2004.

GALINDO JUNIOR, Edemilton Alcides; ROCHA, Romeu Dias; SOUZA, Ronierison de. Desenvolvimento de API REST com Spring Boot. *RIOS - Revista Científica do Centro Universitário do Rio São Francisco*, Paulo Afonso, v. 15, n. 29, p. 119-131, jun. 2021. Disponível em: www.publicacoes.unirios.edu.br. Acesso em: 18 fev. 2026.

GUEDES, Gilleanes T. A. **UML 2**: uma abordagem prática. São Paulo: Novatec, 2011.

HEUSER, Carlos Alberto. **Projeto de banco de dados**. 6. ed. Porto Alegre: Bookman, 2009.

IGNÁCIO, Sergio Aparecido. *Importância da Estatística para o Processo de Conhecimento e Tomada de Decisão*. Revista Paranaense de Desenvolvimento - RPD, [S. l.], n. 118, p. 175–192, 2012. Disponível em: <https://ipardes.emnuvens.com.br/revistaparanaense/article/view/89>. Acesso em: 13 out. 2025.

JACOBSON, Ivar; **BOOCH**, Grady; **RUMBAUGH**, James. **The unified software development process**. Reading: Addison-Wesley, 1999.

JETBRAINS. 25 coisas que nós amamos no Java. [S. l.], 25 maio 2020. Disponível em: https://blog.jetbrains.com/pt-br/2020/05/25/25_coisas_que_nos_amamos_no_java/. Acesso em: 18 fev. 2026.

LARMAN, Craig. **Utilizando UML e padrões**: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo. 3. ed. Porto Alegre: Bookman, 2007.

MARINHO, Lucas Frota; GARCIA, Mario Angel Praia. Desenvolvimento de softwares usando ferramentas front-end baseadas em componentes. 2021. 52 f. Trabalho de Conclusão de Curso (Graduação em Engenharia de Software) – Centro Universitário Fametro, Manaus, 2021.

NETO, Miguel; PINHEIRO, António C.; COELHO, José Castro. *Gestão da Empresa Agrícola no Século XXI. Manual III - Tecnologias de Informação e Comunicação na Gestão da Empresa Agrícola*. Évora: Universidade de Évora, 2007. Disponível em: <https://dspace.uevora.pt/rdpc/handle/10174/2004>. Acesso em: 13/10/2025.

PAULA FILHO, Wilson de Pádua. **Engenharia de software**: fundamentos, métodos e padrões. 3. ed. Rio de Janeiro: LTC, 2009.

PMI - PROJECT MANAGEMENT INSTITUTE. **Guia PMBOK®: Um Guia para o Conjunto de Conhecimentos em Gerenciamento de Projetos**, Sétima edição, Pennsylvania: PMI, 2021.

PONUTHORAI, Prem Kumar; LOELIGER, Jon. **Version Control with Git**. 3. ed. Sebastopol: O'Reilly Media, 2022.

PRESSMAN, Roger S.; **MAXIM**, Bruce R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

ROCHA, Germanno Correia. Avaliação de acessibilidade visual em duas ferramentas da engenharia de requisitos: Astah e Bizagi. 2021. 56 f. Trabalho de Conclusão de Curso (Graduação em Engenharia de Software) – Universidade Federal do Ceará, Russas, 2021. Disponível em: <http://repositorio.ufc.br/handle/riufc/60963>. Acesso em: 19 fev. 2026.

SANTOS, B. C. *A importância da gestão financeira para o pequeno produtor agrícola*. 2022. 33 f. Trabalho de Conclusão de Curso (Graduação em Agronomia) – Centro Universitário Luterano de Palmas, Palmas, 2022.

SARIMBEKOV, A. *et al.* Characteristics of dynamic JVM languages. In: Proceedings of the 7th ACM workshop on Virtual machines and intermediate languages (VMIL '13). New York: Association for Computing Machinery, 2013. p. 11–20. Disponível em: <https://doi.org/10.1145/2542142.2542144>. Acesso em: 26/03/2026.

SILVA, F. A. S.; IGLESIAS, W. A. N.; BOER, M. T. Utilização da prototipagem e criação do front-end pelo Figma para um aplicativo móvel de pets. 2024. Artigo de Graduação (Tecnologia em Sistemas para Internet) – Faculdade de Tecnologia Prof. José Camargo, Jales, 2024. Disponível em: <https://ric.cps.sp.gov.br/handle/123456789/30692>. Acesso em: 19 fev. 2026.

SOMMERVILLE, Ian. **Engenharia de software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

WESTERVELD, Dave. API Testing and Development with Postman: a practical guide to creating, testing, and managing APIs for automated software testing. Birmingham: Packt Publishing, 2021. 226 p. Disponível em: <https://reference-global.com/book/9781800565739?tab=resources>. Acesso em: 19 fev.

2026.

ZAMBALDE, André Luiz *et al.* Tecnologia da informação no agronegócio. *Em: Agricultura Digital: pesquisa, desenvolvimento e inovação nas cadeias produtivas.* Brasília: Embrapa, 2011. p. 55-80.

8. GLOSSÁRIO

9. APÊNDICE

10. ANEXOS